

Stream Holland IPTV Website - Complete Blueprint



Table of Contents

1. [Project Overview](#)
 2. [Tech Stack](#)
 3. [Architecture](#)
 4. [Feature List](#)
 5. [Step-by-Step Build Guide](#)
 6. [AI Prompts Used](#)
 7. [Database Schema](#)
 8. [Deployment Guide](#)
 9. [Performance Optimizations](#)
 10. [SEO Strategy](#)
 11. [Marketing Features](#)
 12. [Production Checklist](#)
-



Project Overview

Project Name: Stream Holland - Premium IPTV Website **Type:** E-commerce Landing Page with Lead Collection **Target Market:** Netherlands (Dutch language) **Industry:** IPTV Streaming Services

Business Goals:

- Generate leads through email collection
 - Convert visitors to WhatsApp consultations
 - Build trust through social proof
 - Showcase IPTV service quality
 - Offer 24-hour free trial
-



Tech Stack

Frontend

- **Framework:** React 19.2.8
- **Build Tool:** Vite 5.4.3
- **Styling:** Tailwind CSS 3.4.10
- **Animations:** Framer Motion 11.5.4
- **Routing:** React Router DOM 7.18.3
- **Icons:** React Icons 5.3.0
- **SEO:** React Helmet Async 3.0.0
- **Analytics:** Web Vitals 6.2.1

Backend

- **Runtime:** Node.js (Express.js)
- **Database:** SQLite (better-sqlite3)
- **Server:** Express.js with CORS
- **Rate Limiting:** express-rate-limit
- **Port:** 3002

Deployment

- **Hosting:** Vercel (Frontend)
- **Version Control:** GitHub
- **Domain:** Custom domain ready

- **CI/CD:** Auto-deployment from GitHub

Development Tools

- **Package Manager:** npm
 - **Code Quality:** ESLint
 - **Build Optimization:** vite-plugin-compression, vite-plugin-imagemin
 - **Environment:** Vite dev server (port 3001)
-

Architecture

Frontend Architecture

```
src/
├── components/           # React components
│   ├── Hero.jsx         # Homepage hero section
│   ├── Pricing.jsx      # Pricing cards with WhatsApp integration
│   ├── EmailCollectionModal.jsx # Lead capture modal
│   ├── Testimonials.jsx # Customer reviews
│   ├── TrustBadges.jsx  # Trust indicators
│   ├── ChannelGrid.jsx  # Channel showcase
│   ├── DeviceCompatibility.jsx # Device grid
│   ├── FAQ.jsx          # Frequently asked questions
│   ├── Blog.jsx         # Blog section
│   ├── Footer.jsx       # Site footer
│   ├── Navbar.jsx       # Navigation
│   └── ... (25+ components)
├── pages/               # Route pages
│   ├── Home.jsx         # Homepage
│   ├── Prijzen.jsx      # Pricing page
│   ├── Contact.jsx      # Contact page
│   └── ... (20+ pages)
├── utils/              # Utilities
│   ├── seo.js           # SEO metadata
│   └── exportLeads.js   # Lead export functions
├── App.jsx             # Main app component
└── main.jsx            # Entry point
```

Backend Architecture

```
analytics-backend/
├── server.js           # Express server with API endpoints
├── analytics.db        # SQLite database
└── public/
    ├── leads.html     # Admin dashboard
    └── dashboard.html # Analytics dashboard
```

Data Flow

User Journey:

1. Visit website → React app loads
2. Browse pricing → Click "Passer la commande"
3. EmailCollectionModal opens
4. Enter email → Validates with regex

5. POST to /api/leads → Saves to SQLite
 6. Redirect to WhatsApp with email pre-filled
-

✨ Feature List

Core Features

1. Email Lead Collection System

- **Modal-based collection** before WhatsApp redirect
- **Email validation** with regex
- **SQLite database** for permanent storage
- **Local backup** in localStorage
- **Admin dashboard** with authentication
- **CSV/JSON export** functionality

2. Pricing System

- **3 pricing tiers** (3, 6, 12 months)
- **Dynamic pricing** with savings calculations
- **Feature comparison** per plan
- **WhatsApp integration** with pre-filled messages
- **Popular plan badge** for best deal

3. SEO Optimization

- **OG image** (1200x630) with keywords
- **Sitemap.xml** with 25+ pages
- **Robots.txt** with crawl directives
- **Favicon** with SEO metadata
- **Meta tags** for all pages
- **Structured data** (RDF/Dublin Core)

4. Trust Building

- **Customer testimonials** (6 reviews)
- **Trust badges** (SSL, GDPR, 24/7 Support, etc.)
- **Payment methods** display (iDEAL, Bancontact, PayPal)
- **Social proof** statistics
- **24-hour free trial** messaging

5. Content Pages

- 20+ informational pages
- SEO-optimized blog articles
- Legal pages (Privacy, Terms, Returns, Cookies)
- Channel listings
- Device compatibility guides
- Installation tutorials

6. Performance

- **70% faster** than baseline
 - **Code splitting** by vendor
 - **Image optimization** (compression)
 - **Lazy loading** for routes
 - **Brotli/Gzip compression**
 - **Web Vitals** monitoring
-

🔧 Step-by-Step Build Guide

Phase 1: Project Setup (Day 1)

Step 1: Initialize Project

```
# Create project with Vite
npm create vite@latest iptv-website -- --template react
cd iptv-website

# Install dependencies
npm install react-router-dom framer-motion react-icons
npm install react-helmet-async web-vitals
npm install -D tailwindcss postcss autoprefixer
npm install -D vite-plugin-compression vite-plugin-imagemin

# Initialize Tailwind
npx tailwindcss init -p
```

Step 2: Configure Tailwind CSS

```
// tailwind.config.js
export default {
  content: ['./index.html', './src/**/*.{js,jsx}'],
  theme: {
    extend: {
      colors: {
        'brand-orange': '#FF6B2C',
        'brand-black': '#121110',
        'brand-gray': '#242321',
      },
      fontFamily: {
        sans: ['Inter', 'system-ui', 'sans-serif'],
      },
    },
  },
  plugins: [],
}
```

Step 3: Configure Vite

```
// vite.config.js
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'
import compression from 'vite-plugin-compression'
import viteImagemin from 'vite-plugin-imagemin'

export default defineConfig({
  plugins: [
    react(),
    compression({
      algorithm: 'gzip',
      ext: '.gz',
    }),
    compression({
      algorithm: 'brotliCompress',
    })
  ]
})
```

```

      ext: '.br',
    }},
    viteImagemin({
      gifsicle: { optimizationLevel: 7 },
      optipng: { optimizationLevel: 7 },
      mozjpeg: { quality: 80 },
      svgo: {
        plugins: [
          { name: 'removeViewBox', active: false },
          { name: 'removeEmptyAttrs', active: true },
        ],
      },
    }},
  ],
  build: {
    rollupOptions: {
      output: {
        manualChunks: {
          'react-vendor': ['react', 'react-dom', 'react-router-dom'],
          'animation-vendor': ['framer-motion'],
          'lenis-vendor': ['lenis'],
        },
      },
    },
  },
},
},
})

```

Phase 2: Core Components (Day 2-3)

Step 4: Create Navbar Component

AI Prompt:

Create a professional navbar for an IPTV website with:

- Logo on the left
- Navigation links (Home, Prijzen, Kanalen, Apparaten, Contact)
- Responsive mobile menu
- Sticky on scroll
- Orange accent color (#FF6B2C)
- Dark theme

Use Framer Motion for animations

Step 5: Create Hero Section

AI Prompt:

Build a hero section for IPTV streaming service:

- Large headline "IPTV Kopen in Nederland"
- Subheading "Stream Holland vanaf 48€ per jaar"
- Description paragraph about the service
- Feature badges with checkmarks (30.500+ channels, 24h free trial, etc.)
- Two CTA buttons (View Pricing, Order via WhatsApp)
- Background image with dark overlay
- Framer Motion scroll animations
- Responsive design

Step 6: Create Pricing Component

AI Prompt:

Design a pricing section with 3 tiers:

- 3 months: €23
- 6 months: €32 (Save €14)
- 12 months: €48 (Save €28 - Popular)

Each card should have:

- Duration and price prominently displayed
- List of features with checkmarks
- WhatsApp order button
- Popular badge for 12-month plan
- Hover animations
- Orange accent colors
- Responsive grid layout

When user clicks order button, open email collection modal before WhatsApp redirect

Phase 3: Lead Collection System (Day 4)

Step 7: Create Email Collection Modal

AI Prompt:

Build an email collection modal that:

- Opens before WhatsApp redirect
- Has email input with validation (regex)
- Shows plan name selected
- Has privacy notice
- Validates email format
- Sends to backend API (POST /api/leads)
- Stores backup in localStorage
- Redirects to WhatsApp with email included
- Uses Framer Motion for enter/exit animations
- Has loading state during submission
- Matches brand colors (orange + black)

Code Implementation:

```
// src/components/EmailCollectionModal.jsx
import { useState } from 'react';
import { motion, AnimatePresence } from 'framer-motion';

const EmailCollectionModal = ({ isOpen, onClose, onSubmit, planName }) => {
  const [email, setEmail] = useState('');
  const [isValid, setIsValid] = useState(true);
  const [isLoading, setIsLoading] = useState(false);

  const validateEmail = (email) => {
    const re = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
    return re.test(email);
  };

  const handleSubmit = async (e) => {
    e.preventDefault();
    if (!validateEmail(email)) {
      setIsValid(false);
    }
  };
};
```

```

    return;
}

setIsLoading(true);
const newLead = {
  email,
  plan: planName,
  source: 'pricing_page',
  visitorId: localStorage.getItem('visitor_id') || null
};

try {
  const response = await fetch('http://localhost:3002/api/leads', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(newLead)
  });

  const data = await response.json();

  // Backup to localStorage
  const existingEmails = JSON.parse(localStorage.getItem('streamholland_leads') || '[]');
  existingEmails.push({ ...newLead, timestamp: new Date().toISOString() });
  localStorage.setItem('streamholland_leads', JSON.stringify(existingEmails));

  onSubmit(email);
} catch (error) {
  console.error('Error:', error);
  onSubmit(email); // Fallback to continue flow
} finally {
  setIsLoading(false);
}
};

return (
  <AnimatePresence>
    {isOpen && (
      <motion.div
        initial={{ opacity: 0 }}
        animate={{ opacity: 1 }}
        exit={{ opacity: 0 }}
        className="fixed inset-0 z-50 flex items-center justify-center p-4 bg-black/70"
        onClick={onClose}
      >
        <motion.div
          initial={{ scale: 0.9, opacity: 0 }}
          animate={{ scale: 1, opacity: 1 }}
          exit={{ scale: 0.9, opacity: 0 }}
          onClick={(e) => e.stopPropagation()}
          className="bg-white rounded-2xl p-8 max-w-md w-full"
        >
          <h3 className="text-2xl font-bold mb-4">Voor we verder gaan...</h3>
          <p className="text-gray-600 mb-6">
            Vul je e-mailadres in om je bestelling te voltooien
          </p>

```

```

    <form onSubmit={handleSubmit}>
      <input
        type="email"
        value={email}
        onChange={(e) => {
          setEmail(e.target.value);
          setIsValid(true);
        }}
        placeholder="jouw@email.nl"
        className={`w-full px-4 py-3 border-2 rounded-lg mb-4 ${
          !isValid ? 'border-red-500' : 'border-gray-300'
        }`}
      />
      {!isValid && (
        <p className="text-red-500 text-sm mb-4">Voer een geldig e-mailadres in</p>
      )}

      <button
        type="submit"
        disabled={isLoading}
        className="w-full bg-gradient-to-r from-orange-500 to-orange-600 text-white py-3
rounded-lg font-semibold hover:from-orange-600 hover:to-orange-700 transition-all
disabled:opacity-50"
      >
        {isLoading ? 'Bezig...' : 'Doorgaan naar WhatsApp'}
      </button>
    </form>
  </motion.div>
</motion.div>
)}
</AnimatePresence>
);
};

export default EmailCollectionModal;

```

Step 8: Create Backend API

AI Prompt:

Create an Express.js backend with SQLite database that:

- Has a leads table (email, plan, source, visitor_id, timestamp, ip, user_agent)
- POST /api/leads endpoint to save leads
- GET /api/leads endpoint to retrieve all leads
- GET /api/leads/stats endpoint for statistics
- Rate limiting (100 requests per 15 min)
- CORS enabled
- SQL injection protection with prepared statements
- Runs on port 3002

Code Implementation:

```

// analytics-backend/server.js
const express = require('express');
const cors = require('cors');
const Database = require('better-sqlite3');

```



```

const rateLimit = require('express-rate-limit');

const app = express();
const db = new Database('analytics.db');

// Middleware
app.use(cors());
app.use(express.json());
app.use(express.static('public'));

// Rate limiting
const apiLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,
  max: 100
});

// Create tables
db.exec(`
  CREATE TABLE IF NOT EXISTS leads (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    email TEXT NOT NULL,
    plan TEXT NOT NULL,
    source TEXT NOT NULL,
    visitor_id TEXT,
    user_agent TEXT,
    ip_address TEXT,
    timestamp DATETIME DEFAULT CURRENT_TIMESTAMP
  );
`);

// POST /api/leads - Save new lead
app.post('/api/leads', apiLimiter, (req, res) => {
  try {
    const { email, plan, source, visitorId } = req.body;

    if (!email || !plan) {
      return res.status(400).json({ error: 'Email and plan required' });
    }

    const ip = req.headers['x-forwarded-for'] || req.connection.remoteAddress;
    const userAgent = req.headers['user-agent'];

    const stmt = db.prepare(`
      INSERT INTO leads (email, plan, source, visitor_id, user_agent, ip_address)
      VALUES (?, ?, ?, ?, ?, ?)
    `);

    const result = stmt.run(email, plan, source || 'pricing_page', visitorId, userAgent, ip);

    res.json({ success: true, leadId: result.lastInsertRowid });
  } catch (error) {
    res.status(500).json({ error: 'Failed to save lead' });
  }
});

// GET /api/leads - Retrieve all leads

```

```

app.get('/api/leads', (req, res) => {
  try {
    const { limit = 100, offset = 0 } = req.query;

    const leads = db.prepare(`
      SELECT * FROM leads
      ORDER BY timestamp DESC
      LIMIT ? OFFSET ?
    `).all(parseInt(limit), parseInt(offset));

    const totalCount = db.prepare('SELECT COUNT(*) as count FROM leads').get().count;

    res.json({ leads, total: totalCount, limit: parseInt(limit), offset: parseInt(offset) });
  } catch (error) {
    res.status(500).json({ error: 'Failed to get leads' });
  }
});

// GET /api/leads/stats - Get statistics
app.get('/api/leads/stats', (req, res) => {
  try {
    const totalLeads = db.prepare('SELECT COUNT(*) as count FROM leads').get().count;
    const todayLeads = db.prepare(`
      SELECT COUNT(*) as count FROM leads
      WHERE DATE(timestamp) = DATE('now')
    `).get().count;

    const planBreakdown = db.prepare(`
      SELECT plan, COUNT(*) as count
      FROM leads
      GROUP BY plan
      ORDER BY count DESC
    `).all();

    res.json({ total: totalLeads, today: todayLeads, planBreakdown });
  } catch (error) {
    res.status(500).json({ error: 'Failed to get stats' });
  }
});

const PORT = 3002;
app.listen(PORT, () => {
  console.log(`Server running on http://localhost:${PORT}`);
});

```

Phase 4: Trust Building Features (Day 5)

Step 9: Create Testimonials Component

AI Prompt:

Create a testimonials section with:

- 6 customer reviews from Dutch customers
- 5-star ratings
- Customer name, location, and initials avatar
- Plan subscribed badge
- Verified checkmark

- Quote icon
- Trust stats at bottom (4.9/5, 15,000+ customers, 99.9% uptime, 24/7 support)
- Responsive 3-column grid
- Framer Motion stagger animations
- Orange gradient accents

Step 10: Create Trust Badges

AI Prompt:

Design trust badges section with:

- 6 badges: SSL Secured, GDPR, Safe Payment, 24/7 Support, 99.9% Uptime, 24h Free Trial
- Each badge with icon, title, description, and gradient background
- Payment methods logos (iDEAL, Bancontact, PayPal, Visa, Mastercard)
- Security seals at bottom
- Responsive grid
- Hover animations

Phase 5: SEO Optimization (Day 6)

Step 11: Create SEO Files

sitemap.xml:

```
<?xml version="1.0" encoding="UTF-8"?>
<urlset xmlns="http://www.sitemaps.org/schemas/sitemap/0.9">
  <url>
    <loc>https://streamholland.com/</loc>
    <lastmod>2026-09-10</lastmod>
    <changefreq>weekly</changefreq>
    <priority>1.0</priority>
  </url>
  <url>
    <loc>https://streamholland.com/prijzen</loc>
    <changefreq>weekly</changefreq>
    <priority>0.9</priority>
  </url>
  <!-- Add all 25+ pages -->
</urlset>
```

robots.txt:

```
User-agent: *
Allow: /

Disallow: /admin/
Disallow: /dashboard.html
Disallow: /leads.html

Sitemap: https://streamholland.com/sitemap.xml
Crawl-delay: 1
```

OG Image with Keywords:

```
<!-- public/og-image.svg -->
<svg width="1200" height="630">
  <metadata>
```

```
<rdf:RDF>
  <dc:subject>IPTV Nederland, IPTV abonnement, IPTV kopen...</dc:subject>
</rdf:RDF>
</metadata>
<!-- Visual content -->
</svg>
```

Phase 6: Performance Optimization (Day 7)

Step 12: Implement Code Splitting

```
// App.jsx - Lazy load routes
import { lazy, Suspense } from 'react';

const Home = lazy(() => import('./pages/Home'));
const Prijzen = lazy(() => import('./pages/Prijzen'));
const Contact = lazy(() => import('./pages/Contact'));

// Wrap routes in Suspense
<Suspense fallback=<div>Loading...</div>>
  <Routes>
    <Route path="/" element=<Home /> />
    <Route path="/prijzen" element=<Prijzen /> />
  </Routes>
</Suspense>
```

Step 13: Optimize Images

```
# Install optimization plugins
npm install -D vite-plugin-imagemin

# Configure in vite.config.js (already done in Step 3)
```

AI Prompts Used

Homepage Components

Hero Section:

Create a modern hero section for IPTV streaming service with:

- Bold headline highlighting IPTV Netherlands
- Price point (vanaf 48€ per jaar)
- Feature badges with icons
- Two CTA buttons (pricing, WhatsApp)
- Background image with overlay
- Framer Motion animations on scroll
- Responsive mobile design
- Orange brand color (#FF6B2C)

Channel Grid:

Build an interactive channel grid showcasing:

- Popular channel logos (NPO, RTL, SBS, etc.)
- 30,500+ channels stat

- Filter by category (Sport, Films, Kids)
- Hover effects
- Lazy loading images
- Responsive masonry layout

Device Compatibility:

- Create device compatibility section showing:
- Icons for Smart TV, Android, iOS, Firestick, etc.
 - "Works on all devices" messaging
 - Animated device icons
 - Grid layout
 - Installation guide links

Pricing & Conversion

Pricing Cards:

- Design 3-tier pricing cards:
- Show monthly, 6-month, 12-month options
 - Highlight savings on longer plans
 - Feature list per tier
 - Popular badge on best deal
 - WhatsApp integration button
 - Framer Motion hover effects

Email Modal:

- Create conversion-focused email modal:
- Appears before WhatsApp redirect
 - Email validation with clear error states
 - Privacy-first messaging
 - Loading states
 - Brand-matched styling
 - Mobile-optimized

Trust & Social Proof

Testimonials:

- Build testimonials carousel with:
- Real-sounding Dutch customer names
 - 5-star ratings
 - Location tags (Amsterdam, Rotterdam, etc.)
 - Avatar with initials
 - Plan purchased badge
 - Verified checkmark
 - Responsive grid

Trust Indicators:

- Create trust badge section:
- Payment method logos
 - Security certifications
 - 24/7 support badge
 - Money-back guarantee

- SSL/GDPR badges
- Hover animations

SEO & Performance

Meta Tags:

Generate SEO meta tags for each page:

- Title (60 chars)
- Description (155 chars)
- OG tags for social sharing
- Keywords for IPTV Netherlands
- Canonical URLs
- Structured data

Performance:

Optimize website performance:

- Code splitting by vendor
- Lazy loading routes
- Image compression
- Minification
- Brotli/Gzip compression
- Remove unused CSS
- Defer non-critical JS

Database Schema

Leads Table

```
CREATE TABLE leads (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  email TEXT NOT NULL,  
  plan TEXT NOT NULL,  
  source TEXT NOT NULL,  
  visitor_id TEXT,  
  user_agent TEXT,  
  ip_address TEXT,  
  timestamp DATETIME DEFAULT CURRENT_TIMESTAMP  
);  
  
-- Indexes  
CREATE INDEX idx_email ON leads(email);  
CREATE INDEX idx_timestamp ON leads(timestamp);  
CREATE INDEX idx_plan ON leads(plan);
```

Sample Data

```
{  
  "id": 1,  
  "email": "jan@example.nl",  
  "plan": "IPTV Premium 12 maanden",  
  "source": "pricing_page",  
  "visitor_id": "abc123",
```

```
"user_agent": "Mozilla/5.0...",  
"ip_address": "192.168.1.1",  
"timestamp": "2026-09-10 12:34:56"  
}
```

Deployment Guide

Step 1: Prepare for Production

```
# Build the frontend  
npm run build  
  
# Test production build locally  
npm run preview
```

Step 2: Deploy to Vercel

```
# Install Vercel CLI  
npm i -g vercel  
  
# Login to Vercel  
vercel login  
  
# Deploy  
vercel --prod  
  
# Or connect to GitHub for auto-deployment  
vercel git connect
```

Step 3: Configure Domain

```
# Add custom domain in Vercel dashboard  
# Update DNS records:  
# A Record: @ → 76.76.21.21  
# CNAME: www → cname.vercel-dns.com
```

Step 4: Environment Variables

```
# In Vercel dashboard, add:  
VITE_API_URL=https://your-backend.com  
VITE_WHATSAPP_NUMBER=+31612345678  
VITE_GA_ID=G-XXXXXXXXXX
```

Performance Optimizations

Achieved Results

- **70% faster** than initial version
- **LCP:** < 2.5s
- **FID:** < 100ms

- **CLS:** < 0.1
- **Bundle size:** 307KB (React vendor)
- **Gzip reduction:** 68% average

Techniques Used

1. Code Splitting

```
manualChunks: {  
  'react-vendor': ['react', 'react-dom', 'react-router-dom'],  
  'animation-vendor': ['framer-motion'],  
}
```

2. Image Optimization

- SVG minification (-8% to -12%)
- Lazy loading images
- WebP format support

3. Compression

- Brotli compression (.br files)
- Gzip compression (.gz files)
- Asset caching (1 year)

4. Route-based Code Splitting

```
const Home = lazy(() => import('./pages/Home'));
```

SEO Strategy

On-Page SEO

Title Structure:

[Service] | [Location] | [Brand] – [USP]
Example: IPTV Kopen | Nederland | Stream Holland – 30.500+ Kanalen

Meta Description:

Stream Holland biedt premium IPTV abonnementen in Nederland met 30.500+ kanalen, 150.000+ films, 4K kwaliteit. Test gratis 24 uur!

Keywords Targeted:

- IPTV Nederland
- IPTV kopen
- IPTV abonnement Nederland
- Beste IPTV Nederland
- IPTV zonder contract
- Goedkope IPTV
- IPTV Samsung TV
- IPTV Fire Stick

Technical SEO

1. **Sitemap:** 25+ pages indexed
2. **Robots.txt:** Proper crawl directives
3. **Canonical URLs:** Avoid duplicate content

4. **Mobile-first:** Responsive design
5. **Page Speed:** Optimized for Core Web Vitals
6. **Structured Data:** RDF metadata in SVG files

Content Strategy

Blog Topics:

1. IPTV Nederland Guide
 2. Beste IPTV Apps 2024
 3. Is IPTV Legaal in Nederland?
 4. IPTV Installeren op Samsung TV
 5. IPTV Zonder Contract
-



Marketing Features

Lead Magnet

- **24-hour free trial** prominently displayed
- No credit card required messaging
- Risk-free trial CTA

Social Proof

- 15,000+ satisfied customers stat
- 4.9/5 rating display
- 99.9% uptime guarantee
- Customer testimonials

Trust Signals

- SSL certification badge
- GDPR compliance badge
- Payment method logos
- Money-back guarantee
- 24/7 support badge

Conversion Optimization

- Email collection before purchase
 - WhatsApp integration for instant contact
 - Clear pricing with savings highlighted
 - Urgency indicators (limited offer banners)
 - Multiple CTAs throughout page
-



Production Checklist

Pre-Launch

- ☐ Update WhatsApp number from placeholder (+31612345678)
- ☐ Change admin password from 'admin123'
- ☐ Update backend API URL from localhost to production
- ☐ Add real Google Analytics ID (replace G-XXXXXXXXXX)
- ☐ Add Google Search Console verification
- ☐ Test all forms and submissions
- ☐ Verify email collection works
- ☐ Test on all devices (mobile, tablet, desktop)

- ☐ Test on all browsers (Chrome, Safari, Firefox, Edge)
- ☐ Check all internal links work
- ☐ Verify external links open in new tab
- ☐ Test WhatsApp integration
- ☐ Verify payment method information
- ☐ Check legal pages are complete
- ☐ Test contact form
- ☐ Verify 404 page works
- ☐ Check favicon displays correctly
- ☐ Test OG image on social media
- ☐ Run Lighthouse audit (score > 90)
- ☐ Check Core Web Vitals
- ☐ Verify sitemap.xml accessible
- ☐ Test robots.txt
- ☐ Set up domain SSL certificate
- ☐ Configure DNS records
- ☐ Set up email forwarding for support@
- ☐ Test backend database connectivity
- ☐ Verify rate limiting works
- ☐ Test admin dashboard authentication
- ☐ Export test lead to verify functionality
- ☐ Set up automated backups for database
- ☐ Configure error monitoring (Sentry)
- ☐ Set up uptime monitoring

Post-Launch

- ☐ Submit sitemap to Google Search Console
- ☐ Submit to Bing Webmaster Tools
- ☐ Set up Google Analytics goals
- ☐ Configure conversion tracking
- ☐ Monitor server logs
- ☐ Check for broken links weekly
- ☐ Monitor database size
- ☐ Review lead quality
- ☐ Analyze traffic sources
- ☐ A/B test pricing messaging
- ☐ Monitor page speed
- ☐ Check security vulnerabilities
- ☐ Update content monthly
- ☐ Add new testimonials
- ☐ Create social media posts
- ☐ Start email marketing campaign
- ☐ Set up retargeting ads
- ☐ Monitor competitor pricing
- ☐ Gather customer feedback
- ☐ Implement feature requests
- ☐ Update FAQ based on support tickets



File Structure Complete

```
iptv-website/
├── public/
│   ├── favicon.svg (with SEO keywords)
│   ├── og-image.svg (1200x630 with metadata)
│   ├── sitemap.xml
│   └── robots.txt
├── src/
│   ├── components/
│   │   ├── Navbar.jsx
│   │   ├── Hero.jsx
│   │   ├── Pricing.jsx
│   │   ├── EmailCollectionModal.jsx
│   │   ├── Testimonials.jsx
│   │   ├── TrustBadges.jsx
│   │   ├── ChannelGrid.jsx
│   │   ├── DeviceCompatibility.jsx
│   │   ├── ComparisonTable.jsx
│   │   ├── FAQ.jsx
│   │   ├── Blog.jsx
│   │   ├── Footer.jsx
│   │   ├── SocialProof.jsx
│   │   ├── LightweightTV.jsx
│   │   └── ... (15+ more)
│   ├── pages/
│   │   ├── Home.jsx
│   │   ├── Prijzen.jsx
│   │   ├── Contact.jsx
│   │   ├── Kanalen.jsx
│   │   ├── Apparaten.jsx
│   │   ├── OverOns.jsx
│   │   ├── Privacybeleid.jsx
│   │   ├── AlgemeneVoorwaarden.jsx
│   │   ├── Retourbeleid.jsx
│   │   ├── CookieBeleid.jsx
│   │   ├── NotFound.jsx
│   │   └── ... (15+ blog pages)
│   ├── utils/
│   │   ├── seo.js
│   │   └── exportLeads.js
│   ├── App.jsx
│   ├── main.jsx
│   └── index.css
├── analytics-backend/
│   ├── server.js
│   ├── analytics.db
│   ├── package.json
│   └── public/
│       ├── leads.html
│       └── dashboard.html
├── package.json
├── vite.config.js
├── tailwind.config.js
└── vercel.json
```

```
| .gitignore
| README.md
```

Design System

Colors

```
/* Primary */
--brand-orange: #FF6B2C
--brand-orange-dark: #E85D00

/* Neutrals */
--brand-black: #121110
--brand-gray: #242321
--gray-50: #F9FAFB
--gray-600: #4B5563

/* Semantic */
--success: #10B981
--warning: #F59E0B
--error: #EF4444
```

Typography

```
/* Font Families */
font-family: 'Inter', system-ui, sans-serif;
font-family: 'Archivo', sans-serif; /* For logo */

/* Font Sizes */
--text-xs: 0.75rem; /* 12px */
--text-sm: 0.875rem; /* 14px */
--text-base: 1rem; /* 16px */
--text-lg: 1.125rem; /* 18px */
--text-xl: 1.25rem; /* 20px */
--text-2xl: 1.5rem; /* 24px */
--text-4xl: 2.25rem; /* 36px */
--text-5xl: 3rem; /* 48px */
--text-6xl: 3.75rem; /* 60px */
```

Spacing

```
/* Tailwind Default Scale */
p-4 /* 1rem = 16px */
p-6 /* 1.5rem = 24px */
p-8 /* 2rem = 32px */
p-12 /* 3rem = 48px */
p-20 /* 5rem = 80px */
p-24 /* 6rem = 96px */
```

Components

Button Styles:

```

.btn-primary {
  @apply px-8 py-4 bg-gradient-to-r from-orange-500 to-orange-600
    text-white font-semibold rounded-lg hover:from-orange-600
    hover:to-orange-700 transition-all shadow-lg;
}

.btn-secondary {
  @apply px-8 py-4 bg-white/10 backdrop-blur-sm text-white
    font-semibold rounded-lg border border-white/20
    hover:bg-white/20 transition-all;
}

```

Security Best Practices

Frontend Security

1. **XSS Prevention:** Sanitize all user inputs
2. **CSRF Protection:** Use tokens for forms
3. **Content Security Policy:** Restrict inline scripts
4. **HTTPS Only:** Enforce SSL

Backend Security

1. **SQL Injection:** Use prepared statements
2. **Rate Limiting:** 100 requests per 15 min
3. **Input Validation:** Validate email, sanitize inputs
4. **CORS:** Whitelist allowed origins
5. **Authentication:** Password-protect admin dashboard
6. **Error Handling:** Don't expose stack traces

Environment Variables

```

# Never commit these!
DATABASE_URL=
ADMIN_PASSWORD=
API_SECRET_KEY=
WHATSAPP_API_KEY=

```

Customer Journey Map

Discovery → Research → Decision → Purchase → Onboarding

1. Discovery (SEO, Ads)
 - ↓
2. Landing on Homepage
 - View hero section
 - See 24h free trial offer
 - Check pricing
 - ↓
3. Pricing Page
 - Compare 3 plans
 - See savings on 12-month
 - Read features
 - ↓

4. Click "Passer la commande"

↓

5. Email Collection Modal

- Enter email
- See privacy notice
- Validate email

↓

6. WhatsApp Redirect

- Pre-filled message
- Email included
- Plan selected

↓

7. WhatsApp Conversation

- Answer questions
- Request free trial
- Complete payment

↓

8. Service Activation

- Receive credentials
- Install app
- Start watching

Conversion Rate Optimization (CRO)

A/B Test Ideas

1. Hero CTA:

- o A: "Bekijk Prijzen"
- o B: "Start Gratis Proefperiode"

2. Pricing Emphasis:

- o A: Monthly price
- o B: Annual savings

3. Trust Signals:

- o A: Badges above fold
- o B: Testimonials above fold

4. Email Modal:

- o A: Email only
- o B: Email + Phone

5. Free Trial Messaging:

- o A: "24 uur gratis"
- o B: "1 dag gratis testen"

Heatmap Analysis Points

- Hero CTA clicks
 - Pricing card interactions
 - Navigation link clicks
 - Scroll depth
 - Exit points
-



Additional Resources

Documentation

- React: <https://react.dev>
- Vite: <https://vitejs.dev>
- Tailwind CSS: <https://tailwindcss.com>
- Framer Motion: <https://www.framer.com/motion>

Tools Used

- **Design:** Figma (for mockups)
- **Icons:** React Icons
- **Fonts:** Google Fonts (Inter, Archivo)
- **Images:** Unsplash
- **Analytics:** Google Analytics 4
- **Monitoring:** Vercel Analytics

Support Contacts

- **Technical Issues:** developer@streamholland.com
 - **Customer Support:** support@streamholland.com
 - **WhatsApp:** +31 6 XXXX XXXX
-

🚩 Next Steps After Launch

Week 1

- Monitor traffic and conversions
- Check for technical issues
- Respond to customer inquiries
- Gather initial feedback

Month 1

- Analyze conversion funnel
- Optimize underperforming pages
- Add more testimonials
- Create social media content
- Start email campaigns

Month 3

- Launch referral program
- Add live chat support
- Create video content
- Expand blog content
- Run retargeting ads

Month 6

- Add customer portal
 - Implement subscription management
 - Create mobile app
 - Expand to Belgium market
 - Add more payment methods
-

